Department of Computer Science and Engineering

CSA06 – Design and Analysis of Algorithms

CO2 – Debugging and Optimisation Task

SOC Alert-Triage Module of "SentinelOps"

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Course Outcome Assessed	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems.
Maximum Marks	50
Mode	Individual
Student Reg. No.	192565006
Student Name	Thanigaivel s

(a) Reproduce and Identify the Defects (10 Marks)

Test data: alerts = [(A1,3), (A2,9), (A3,5), (A4,9), (A5,1)]; blocklist = [10.0.0.5, 10.0.0.9, 172.16.4.2]; queries = [10.0.0.9, 10.0.0.5, 8.8.8.8]; log = FAILFAIXFAIL, pattern = FAIL; events = [(2,3), (12,30), (40,50), (5,1), (12,10), (3,4)].

Function	Expected output	Actual output
sort_alerts	[A2:9, A4:9, A3:5, A1:3, A5:1]	[A4:9, A3:5, A3:5, A5:1, A5:1] (A1 and A2 vanish; A3 and A5 are duplicated)
is_blocked	[True, True, False]	[False, False, False]
find_signature	[0, 8]	[0, 4] (8 missed, 4 is false)
closest_logins	((0, 5), 1.4142)	((0, 2), 60.44) (the farthest pair)

Expected closest pair: points 0 = (2,3) and 5 = (3,4) differ by (1,1), so the distance is $\sqrt{2} \approx 1.4142$ km. The next best pairs (0,3) and (3,5) are $\sqrt{13} \approx 3.61$ km.

The nine defects

No.	Function	Faulty line	Nature of the error
1	sort_alerts	while j > 0 and alerts[j][1] < key[1]:	Off-by-one boundary. Index 0 is never compared or shifted. Should be j >= 0.
2	sort_alerts	alerts[j] = key	Wrong position. The key must go in the gap at j + 1. Overwrites an existing alert.
3	is_blocked	return False (inside if ==)	Wrong value. A match must return True.
4	is_blocked	else: return False (inside loop)	Premature exit. Only the first entry is checked. An empty list returns None.

No.	Function	Faulty line	Nature of the error
5	find_signature	for i in range(n - m):	Off-by-one. The last window (start n − m) is never tried. Should be n − m + 1.
6	find_signature	if j == m - 1:	Wrong match test. A mismatch on the last character also leaves j = m − 1, so a false hit is added.
7	closest_logins	min_dist = 0	Wrong start value. No distance is below 0. Should be infinity.
8	closest_logins	for j in range(len(events)):	Pairs the point with itself and repeats pairs (i,j) and (j,i). Should start at i + 1.
9	closest_logins	if d > min_dist:	Wrong comparison. This finds the largest distance. Should be d < min_dist.

(b) Root Causes and Systematic Debugging (12 Marks)

Each function is corrected on its own, and re-tested after each change. Only the faulty lines are changed.

1. sort_alerts (defects 1 and 2)

Root cause: Insertion sort takes alerts[i] as the key, shifts smaller items one place right, then drops the key in the gap. The gap is at j + 1, and the scan must reach index 0.

Buggy trace (the key is placed at j, and j stops at 0):

Pass i	Key	List after the pass
start	–	A1:3, A2:9, A3:5, A4:9, A5:1
1	A2:9	A2:9, A2:9, A3:5, A4:9, A5:1 (A1 lost)
2	A3:5	A2:9, A3:5, A3:5, A4:9, A5:1
3	A4:9	A4:9, A3:5, A3:5, A3:5, A5:1
4	A5:1	A4:9, A3:5, A3:5, A5:1, A5:1

Corrected trace (j >= 0 and key placed at j + 1):

Pass i	Key and shifts	List after the pass
start	–	A1:3, A2:9, A3:5, A4:9, A5:1
1	A2:9. A1 (3 < 9) shifts. j = −1, place at 0.	A2:9, A1:3, A3:5, A4:9, A5:1
2	A3:5. A1 shifts. A2 (9 < 5 is false), stop. Place at 1.	A2:9, A3:5, A1:3, A4:9, A5:1
3	A4:9. A1 and A3 shift. A2 (9 < 9 is false), stop. Place at 1.	A2:9, A4:9, A3:5, A1:3, A5:1

Pass i	Key and shifts	List after the pass
4	A5:1. A1 (3 < 1 is false), stop. Place at 4.	A2:9, A4:9, A3:5, A1:3, A5:1

Why the strict < matters: A2 and A4 both have severity 9. Because 9 < 9 is false, A4 is not moved in front of A2, so arrival order is kept (stable).

Re-test after each fix: fixing only defect 1 gives [A1, A1, A1, A1, A4] (the key is written to index −1, which Python wraps to the last item). Fixing only defect 2 gives [A1, A2, A4, A3, A5] (A1 is stuck in front). With both fixed the result is [A2, A4, A3, A1, A5]. ✓

2. is_blocked (defects 3 and 4)

Root cause: A search must return True on the first match and False only after all entries are checked. The code returns False on a match (defect 3) and also returns False at the first non-match (defect 4), so the function ends after one comparison.

Query	Buggy behaviour	Corrected behaviour
10.0.0.9	Index 0 = 10.0.0.5, not equal, else returns False.	Index 1 matches, returns True.
10.0.0.5	Index 0 matches, returns False.	Index 0 matches, returns True.
8.8.8.8	Index 0 not equal, returns False (right by luck).	All 3 checked, returns False.

Fix: return True inside the if, and put `return False` after the loop. Re-test result: [True, True, False]. ✓ Empty blocklist now returns False (before: None).

3. find_signature (defects 5 and 6)

Root cause: Defect 5 skips the last window. Defect 6 tests j == m − 1 after the inner loop. When the mismatch happens at the last character, j is also m − 1, so the code cannot tell a full match from a last-character mismatch.

Alignment table (log = FAILFAIXFAIL, pattern = FAIL, n = 12, m = 4, so 9 windows):

Start i	Window	Result (correct)	Buggy behaviour
0	FAIL	Match, 4 comparisons	Hit (correct)
1	AILF	Mismatch at j = 0	No hit
2	ILFA	Mismatch at j = 0	No hit
3	LFAI	Mismatch at j = 0	No hit
4	FAIX	Mismatch at j = 3 (X ≠ L)	**False hit** (j = 3 = m − 1)
5	AIXF	Mismatch at j = 0	No hit
6	IXFA	Mismatch at j = 0	No hit
7	XFAI	Mismatch at j = 0	No hit
8	FAIL	Match, 4 comparisons	**Missed** (range stops at 7)

Fix: use range(n − m + 1) and a for-else, so that the else branch runs only when no break happened. Re-test after each fix: fixing only defect 5 gives [0, 4, 8] (false hit remains); fixing only defect 6 gives [0] (the hit at 8 is still missed). With both fixed the result is [0, 8]. ✓

4. closest_logins (defects 7, 8 and 9)

Root cause: Finding a minimum needs a start value larger than any distance (infinity), the comparison <, and pairs of different points. The code starts at 0 and uses >, so it keeps the largest distance. The pair (0,2) is the farthest: $\sqrt{(38^2 + 47^2)} = \sqrt{3653} \approx 60.44$ km.

Re-test after each fix:

Version	Result
Only defect 9 fixed (min_dist = 0)	(None, 0): nothing is smaller than 0
Defects 7 and 9 fixed (self pairs remain)	((0, 0), 0.0): a point paired with itself

Version	Result
Defects 7, 8 and 9 fixed	((0, 5), 1.4142) ✓

Boundary tests (after all corrections)

Test	Expected	Result
Empty blocklist	False	Pass
Pattern at the very end (XXFAIL)	[2]	Pass
Pattern longer than the log (AB, ABC)	[]	Pass
Equal severities (a5, b5, c5)	Same order	Pass
Fewer than two events ([] and [(1,1)])	(None, None)	Pass
Overlapping matches (AAAA, AA)	[0, 1, 2]	Pass
Empty pattern	[] (no crash)	Pass
Duplicate location events	Distance 0.0	Pass

Extra safety: the original code crashes with an UnboundLocalError for an empty pattern (j is never set). The fixed code returns [] for it.

(c) Optimise the Corrected Module (10 Marks)

1. Sorting alerts

- **Insertion sort on nearly sorted data:** suitable. It does about n − 1 + (number of inversions) comparisons, so it is nearly linear if few alerts are out of place.
- **But for 50,000 random alerts it is not suitable:** about $n^2/4$ comparisons (hundreds of millions). Reversed input is the worst case, $n(n-1)/2$.
- **Better approach:** Python's built-in sort (Timsort) with key = severity and reverse = True. It is stable, takes O(n log n) in the worst case and close to O(n) on nearly sorted data, so it is good in both situations.

2. Repeated blocklist lookups

- A list scan costs up to m comparisons for every query, so Q queries cost O(Q·m).
- Build a set once with build_blocklist (O(m)). Each lookup then costs O(1) on average, so the total is O(m + Q).
- Output is identical: True if the IP is in the blocklist, else False.

3. Signature scan

- Return [] at once if the pattern is empty or longer than the log (no windows to scan).
- Try only the n − m + 1 legal windows, and stop each window at its first mismatch (break).
- Use for-else, so there is no extra check after the inner loop, and no character is compared twice.

4. Closest pair

- **Remove self and repeated pairs:** j starts at i + 1. The pairs fall from n^2 to $n(n-1)/2$.
- **Remove square roots:** compare squared distances (a smaller square means a smaller distance). Use one sqrt for the final answer.
- **Prune:** sort the points by x. For a point, scan the next points in x order. When the x-gap alone is larger than the best distance ($dx^2 > best^2$), stop, because every later point is even farther.

- **Same output:** the pruning test uses > (not ≥), so equal-distance pairs are still examined. Ties choose the smallest (i, j), exactly like the corrected double loop. This was checked against the brute-force version on 200 random sets full of ties.

(d) Analyse Before and After (10 Marks)

Operation counts and time complexity

Function	Before (corrected brute force)	After (optimised)	Gain
sort_alerts	Comparisons: best $n - 1$ (sorted), average $\approx n^2/4$, worst $n(n-1)/2$. $\Theta(n^2)$.	$\Theta(n \log n)$ worst, about n on nearly sorted.	50,000 alerts: $\approx 6 \times 10^8$ down to $\approx 8 \times 10^5$
is_blocked (Q queries, m entries)	Best 1, worst m per query. Total $O(Q \cdot m)$.	Build O(m), then O(1) per query. Total $O(m + Q)$.	Large for many queries
find_signature	Windows $n - m + 1$. Best $n - m + 1$ comparisons, worst $m(n - m + 1)$. $\Theta(nm)$ worst.	Same count (already minimal for brute force), with no extra tests.	Correct and no wasted work
closest_logins	n^2 distances and n^2 sqrt (original). Corrected: $n(n-1)/2$ and the same number of sqrt. $\Theta(n^2)$.	n log n sort + pruned scan; squared distances; 1 sqrt. Worst case still $\Theta(n^2)$.	8,000 events: 3.2×10^7 down to about 8.7×10^3

Measured: closest_logins distance computations

Random points in a 1000 km × 1000 km area (fixed random seed).

Events n	Original (n^2)	Corrected $n(n-1)/2$	Optimised (measured)	Square roots (optimised)
1,000	1,000,000	499,500	579	1
2,000	4,000,000	1,999,000	3,214	1
4,000	16,000,000	7,998,000	4,825	1

Events n	Original (n^2)	Corrected $n(n-1)/2$	Optimised (measured)	Square roots (optimised)
8,000	64,000,000	31,996,000	8,674	1

Does it agree with the analysis? Yes. The corrected loop matches $n(n - 1)/2$ exactly, which is half of n^2. The pruned version needs only a few thousand computations, near-linear for scattered points. The **worst case** also agrees: if all points share the same x (2,000 points tested), pruning never triggers and it makes 1,999,000 computations, the same as brute force. So the guaranteed bound is still $\Theta(n^2)$, but typical SOC data is far faster.

Measured: insertion sort comparisons

Severities 1 to 10. Nearly sorted = already sorted (highest first), then 1% of the alerts moved to random places.

Alerts n	Insertion, random	Insertion, nearly sorted	Timsort, random	Timsort, nearly sorted
1,000	231,627	4,175	6,718	1,711
2,000	919,590	12,913	13,815	3,485
5,000	5,616,178	77,866	35,293	7,436
10,000	22,303,397	317,590	70,925	14,099

Agreement: random data grows about 4 times when n doubles (231,627 → 919,590), which is the n^2 law (about $0.22n^2$ because many ties are not inversions). Nearly sorted data costs only a few times n, as predicted by $n - 1$ + inversions. Timsort grows slightly faster than n (n log n) on random data and stays close to n on nearly sorted data. On a reversed list of 1,000 alerts, insertion sort made 499,500 = $n(n - 1)/2$ comparisons, matching the worst-case formula.

Measured: blocklist and signature

- **Blocklist:** m = 1,000 entries and 2,000 queries: the list scan made 1,503,517 comparisons, while the set needs about m + Q = 3,000 operations. Time for 10,000 entries and 5,000 queries: list 0.65 s, set 0.0013 s.
- **Signature, test log:** 18 comparisons for 9 windows (4 + 1 + 1 + 1 + 4 + 1 + 1 + 1 + 4).
- **Signature, random DNA text of 100,000 characters, 10-character pattern:** 133,591 comparisons, about 1.3n, close to the n best case because most windows fail at the first or second character.
- **Signature, worst case** (text = 100,000 A's, pattern = 49 A's + B): 4,997,550 comparisons = m(n − m + 1), exactly the worst-case formula.

(e) Production-Quality Code (8 Marks)

The complete module with docstrings, comments and the test harness is below, and also in the file **sentinelops.py**. Running `python sentinelops.py` runs all tests. Result: **34 of 34 tests passed.**

Submission: commit sentinelops.py to your GitHub repository and share the link in Google Classroom (I cannot do this step for you). Commands: `git add sentinelops.py`, `git commit -m "Fix and optimise SentinelOps module"`, `git push`.

```
"""SentinelOps SOC alert-triage module (corrected and optimised).

Functions
---------
sort_alerts      : sort alerts by severity, highest first, stable.
insertion_sort_alerts : corrected insertion sort (kept for reference/tests).
build_blocklist  : turn a list of IPs into a set for O(1) lookups.
is_blocked       : check whether an IP is on the blocklist.
find_signature   : brute-force search for every position of a pattern.
closest_logins   : closest pair of login locations (brute force + pruning).
"""
import math


def insertion_sort_alerts(alerts):
    """Corrected insertion sort, highest severity first, stable.

    Each alert is a tuple (alert_id, severity). Sorts the list in place and
    returns it. Strict '<' keeps equal severities in arrival order.
    Time: O(n) when nearly sorted, O(n^2) on average and worst case.
    """
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:   # FIX: j >= 0 (was j > 0)
            alerts[j + 1] = alerts[j]
```

```
            j -= 1
        alerts[j + 1] = key                    # FIX: j + 1 (was j)
    return alerts


def sort_alerts(alerts):
    """Sort alerts by severity (highest first), keeping arrival order for ties.

    Uses Python's Timsort: O(n log n) worst case and close to O(n) when the
    list is nearly sorted. The sort is stable even with reverse=True.
    Sorts in place and returns the same list (same behaviour as before).
    """
    alerts.sort(key=lambda alert: alert[1], reverse=True)
    return alerts


def build_blocklist(entries):
    """Build a set from the blocklist once: O(m) build, O(1) lookup."""
    return frozenset(entries)


def is_blocked(blocklist, ip):
    """Return True if ip is on the blocklist, else False.

    Works with a list (O(m) per query) or, much faster, with the set made
    by build_blocklist (O(1) per query). An empty blocklist gives False.
    """
    return ip in blocklist


def find_signature(log, pattern):
    """Return every start index where pattern occurs in log (overlaps allowed).

    Brute force: slide the pattern over every legal window and stop each
```

```
    window at the first mismatch. An empty pattern or a pattern longer than
    the log gives [].
    Time: O(n - m + 1) best case, O(m * (n - m + 1)) worst case.
    """
    n, m = len(log), len(pattern)
    if m == 0 or m > n:
        return []
    hits = []
    for i in range(n - m + 1):                  # FIX: n - m + 1 (was n - m)
        for j in range(m):
            if log[i + j] != pattern[j]:
                break                           # mismatch: next window
        else:                                   # FIX: for-else = full match
            hits.append(i)
    return hits


def closest_logins(events):
    """Find the closest pair of login locations.

    events : list of (x, y) points in km.
    Returns ((i, j), distance) with i < j, or (None, None) when fewer than
    two events exist. Ties are resolved to the lexicographically smallest
    (i, j), exactly like the corrected brute-force double loop.

    Optimisations: no self/repeated pairs, squared distances only (one
    sqrt at the end), and points sorted by x so the inner loop stops as soon
    as the x-gap alone is larger than the best distance found so far.
    Time: O(n log n) sort + pruned scan (O(n^2) only in the worst case).
    """
    n = len(events)
    if n < 2:
        return None, None
    order = sorted(range(n), key=lambda k: events[k][0])
```

```
    best_sq = math.inf
    best_pair = None
    for a in range(n):
        ia = order[a]
        xa, ya = events[ia]
        for b in range(a + 1, n):
            ib = order[b]
            dx = events[ib][0] - xa
            if dx * dx > best_sq:               # prune: no later point can win
                break
            dy = events[ib][1] - ya
            d_sq = dx * dx + dy * dy
            pair = (ia, ib) if ia < ib else (ib, ia)
            if d_sq < best_sq or (d_sq == best_sq and pair < best_pair):
                best_sq = d_sq
                best_pair = pair
    return best_pair, math.sqrt(best_sq)


def closest_logins_bruteforce(events):
    """Corrected plain brute force (reference for testing the optimised one)."""
    min_dist = math.inf                         # FIX: was 0
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):       # FIX: j > i (was all j)
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d < min_dist:                    # FIX: '<' (was '>')
                min_dist = d
                pair = (i, j)
    if pair is None:
        return None, None
    return pair, min_dist
```

```
# ---------------------------------------------------------------------
# Test harness
# ---------------------------------------------------------------------
def _check(name, actual, expected):
    """Print PASS/FAIL for one test and return True when it passes."""
    ok = actual == expected
    print(("PASS" if ok else "FAIL"), "-", name, "" if ok else f"-> {actual} != {expected}")
    return ok


def run_tests():
    """Run every test (spec data + boundary cases); return True if all pass."""
    import random
    r = []
    alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
    want = [("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]
    r.append(_check("sort: test data", sort_alerts(list(alerts)), want))
    r.append(_check("sort: insertion sort test data", insertion_sort_alerts(list(alerts)),
want))
    r.append(_check("sort: empty", sort_alerts([]), []))
    r.append(_check("sort: one alert", sort_alerts([("X", 4)]), [("X", 4)]))
    r.append(_check("sort: equal severities keep order",
                    sort_alerts([("a", 5), ("b", 5), ("c", 5)]), [("a", 5), ("b", 5), ("c", 5)]))
    r.append(_check("sort: first element is not lost",
                    sort_alerts([("a", 1), ("b", 2)]), [("b", 2), ("a", 1)]))
    rnd = random.Random(1)
    data = [(f"R{k}", rnd.randint(1, 10)) for k in range(300)]
    r.append(_check("sort: both sorts agree (300 random)",
                    sort_alerts(list(data)), insertion_sort_alerts(list(data))))

    bl = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
    bs = build_blocklist(bl)
```

```
    for q, e in [("10.0.0.9", True), ("10.0.0.5", True), ("172.16.4.2", True), ("8.8.8.8",
False)]:
        r.append(_check(f"blocked: {q} (list)", is_blocked(bl, q), e))
        r.append(_check(f"blocked: {q} (set)", is_blocked(bs, q), e))
    r.append(_check("blocked: empty blocklist", is_blocked([], "1.1.1.1"), False))
    r.append(_check("blocked: empty set", is_blocked(build_blocklist([]), "1.1.1.1"),
False))

    r.append(_check("signature: test data", find_signature("FAILFAIXFAIL", "FAIL"),
[0, 8]))
    r.append(_check("signature: at very end", find_signature("XXFAIL", "FAIL"), [2]))
    r.append(_check("signature: whole log", find_signature("FAIL", "FAIL"), [0]))
    r.append(_check("signature: pattern longer than log", find_signature("AB", "ABC"),
[]))
    r.append(_check("signature: overlapping", find_signature("AAAA", "AA"), [0, 1, 2]))
    r.append(_check("signature: no match", find_signature("ABCD", "XY"), []))
    r.append(_check("signature: empty pattern", find_signature("ABC", ""), []))
    r.append(_check("signature: empty log", find_signature("", "A"), []))
    r.append(_check("signature: no false hit on last-char mismatch",
                    find_signature("FAIX", "FAIL"), []))

    events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
    pair, dist = closest_logins(events)
    r.append(_check("closest: pair", pair, (0, 5)))
    r.append(_check("closest: distance", round(dist, 4), round(math.sqrt(2), 4)))
    r.append(_check("closest: brute force agrees", closest_logins_bruteforce(events),
(pair, dist)))
    r.append(_check("closest: empty", closest_logins([]), (None, None)))
    r.append(_check("closest: one event", closest_logins([(1, 1)]), (None, None)))
    r.append(_check("closest: two events", closest_logins([(0, 0), (3, 4)]), ((0, 1), 5.0)))
    r.append(_check("closest: duplicate location", closest_logins([(1, 1), (9, 9), (1, 1)]),
((0, 2), 0.0)))
    ok = True
    for t in range(200):                    # random tests with many ties
```

```
        pts = [(rnd.randint(0, 15), rnd.randint(0, 15)) for _ in range(rnd.randint(2, 40))]
        if closest_logins(pts) != closest_logins_bruteforce(pts):
            ok = False
    r.append(_check("closest: optimised == brute force (200 random sets)", ok, True))
    print(f"\n{sum(r)} of {len(r)} tests passed")
    return all(r)


if __name__ == "__main__":
    raise SystemExit(0 if run_tests() else 1)
```